Lab 04 - Problema 1

Árbol de expansión mínima como problema de búsqueda

1. Formulación como problema de búsqueda

Se tiene un grafo con varios nodos y conexiones, y cada conexión posee un peso o costo.

Espacio de configuraciones

Las posibles soluciones son todas las formas de elegir conexiones para:

- unir todos los nodos,
- evitar ciclos,
- y usar solo las conexiones necesarias.

En otras palabras, se busca formar un árbol que conecte todo el grafo.

También pueden existir configuraciones parciales, es decir, conjuntos de conexiones que todavía no unen todo el grafo, pero que van en buen camino y no forman ciclos.

Tamaño del espacio de búsqueda

La cantidad de posibles soluciones depende de cómo esté construido el grafo.

Si el grafo tiene muchas conexiones, entonces habrá más maneras de formar un árbol generador.

Descripción de las configuraciones

Cada configuración puede representarse como una lista de conexiones elegidas hasta el momento.

Es decir, en cada paso se van agregando aristas válidas, siempre cuidando que no formen ciclos.

Función de costo

El costo de una solución se obtiene sumando los pesos de todas las conexiones seleccionadas.

Objetivo

El objetivo es encontrar la combinación de conexiones que logre unir todos los nodos con el menor costo total posible.

Componentes de búsqueda

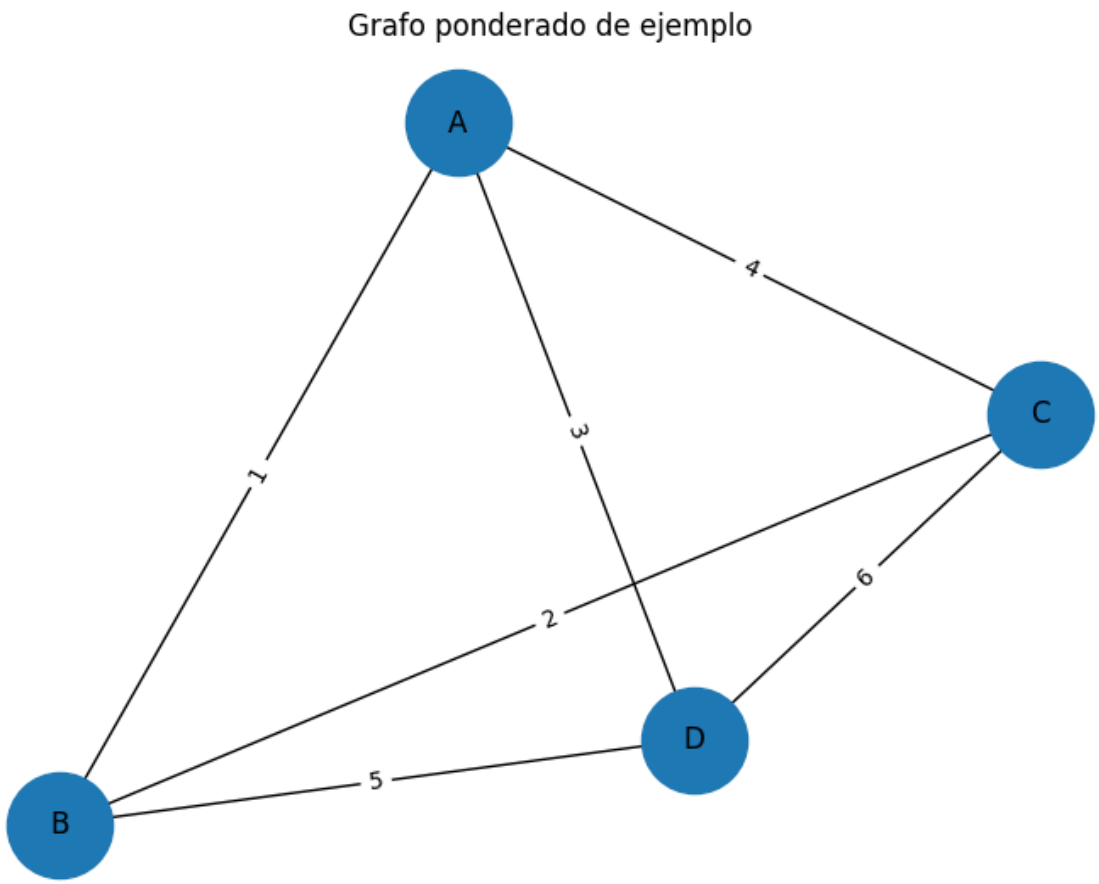
- **Estado inicial:** no se ha elegido ninguna conexión.
- **Acción:** agregar una conexión válida.
- **Restricción:** no formar ciclos.
- **Meta:** conectar todos los nodos usando solo las conexiones necesarias.

In [1]:
```
import networkx as nx
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (8, 5)
```

2. Grafo pequeño de ejemplo

Para que el árbol de búsqueda se vea claro, se usará un grafo pequeño de 4 nodos.

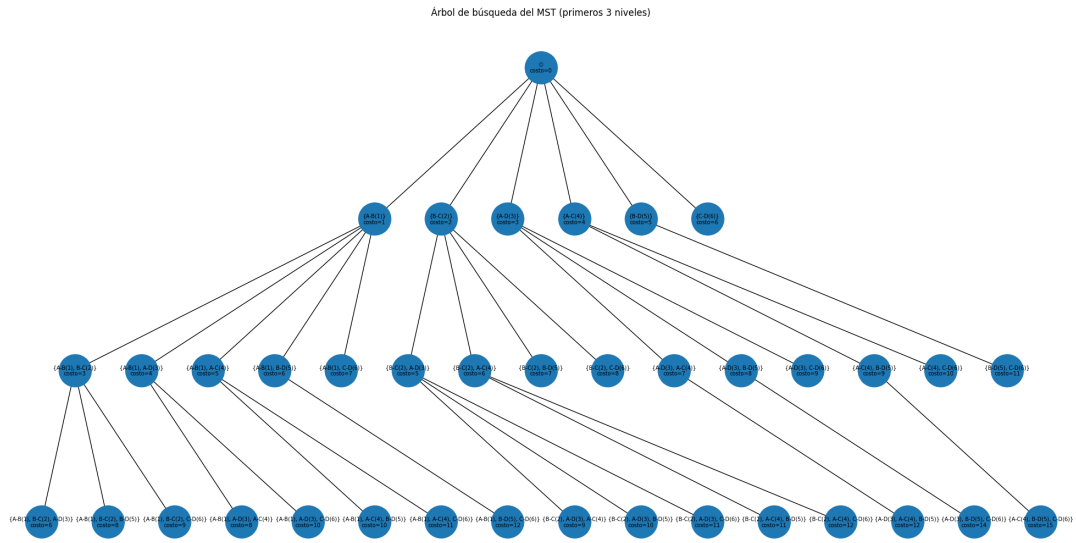
In [2]:
```
# Grafo de ejemplo
vertices = ["A", "B", "C", "D"]
weighted_edges = [
    ("A", "B", 1),
    ("A", "C", 4),
    ("A", "D", 3),
    ("B", "C", 2),
    ("B", "D", 5),
    ("C", "D", 6),
]

G = nx.Graph()
G.add_weighted_edges_from(weighted_edges)

pos = nx.spring_layout(G, seed=7)

plt.figure(figsize=(7, 5))
nx.draw(G, pos, with_labels=True, node_size=2000)
edge_labels = {(u, v): w for u, v, w in weighted_edges}
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels)
plt.title("Grafo ponderado de ejemplo")
plt.axis("off")
plt.show()
```



3. Árbol de búsqueda

In [3]:
```
def edge_label(edge):
    u, v, w = edge
    return f"{u}-{v}({w})"

def forest_from_edges(vertices, edges):
    H = nx.Graph()
    H.add_nodes_from(vertices)
    H.add_weighted_edges_from(edges)
    return H

def is_acyclic(vertices, edges):
    H = forest_from_edges(vertices, edges)
    return nx.is_forest(H)

def state_cost(edges):
    return sum(w for _, _, w in edges)

weighted_edges_sorted = sorted(weighted_edges, key=lambda e: (e[2], e[0], e[1]))

def build_mst_search_tree(vertices, edges, max_depth=3):
    T = nx.DiGraph()
    root = tuple()
    T.add_node(root, level=0, label="∅\ncosto=0")
```

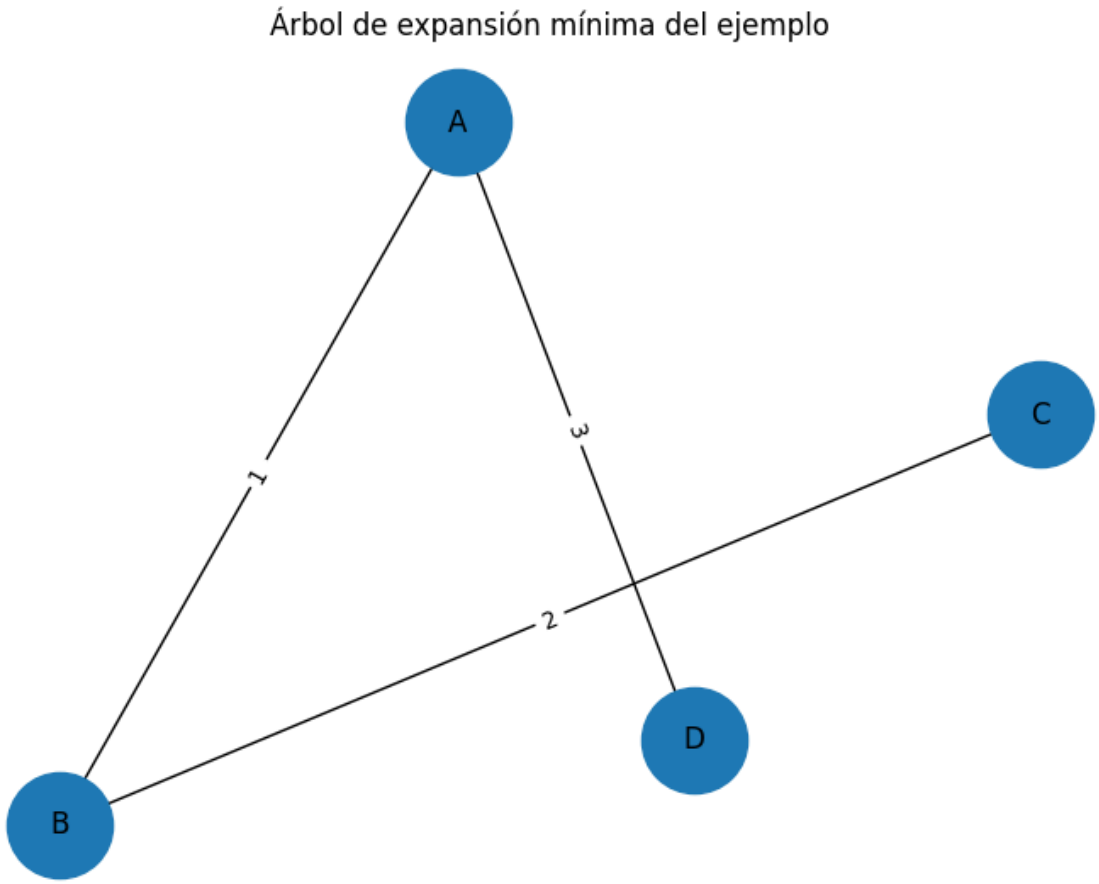
```python
    def expand(state, start_idx, depth):
        if depth == max_depth:
            return

        for idx in range(start_idx, len(edges)):
            edge = edges[idx]
            new_state = tuple(list(state) + [edge])

            if is_acyclic(vertices, new_state):
                if new_state not in T:
                    lbl_edges = ", ".join(edge_label(e) for e in new_state)
                    T.add_node(
                        new_state,
                        level=len(new_state),
                        label=f"{{{lbl_edges}}}\ncosto={state_cost(new_state)}",
                    )

                T.add_edge(state, new_state)
                expand(new_state, idx + 1, depth + 1)

    expand(root, 0, 0)
    return T

search_tree = build_mst_search_tree(vertices, weighted_edges_sorted, max_depth=3)

def layered_positions(G):
    levels = {}
    for node, data in G.nodes(data=True):
        lvl = data["level"]
        levels.setdefault(lvl, []).append(node)

    pos = {}
    for lvl, nodes in levels.items():
        width = len(nodes)
        for i, node in enumerate(nodes):
            x = i - (width - 1) / 2
            y = -lvl
            pos[node] = (x, y)
    return pos

pos_tree = layered_positions(search_tree)
labels_tree = nx.get_node_attributes(search_tree, "label")

plt.figure(figsize=(22, 10))
nx.draw(search_tree, pos_tree, with_labels=False, node_size=1800, arrows=False)
nx.draw_networkx_labels(search_tree, pos_tree, labels=labels_tree, font_size=7)
plt.title("Árbol de búsqueda del MST (primeros 3 niveles)")
plt.axis("off")
plt.show()
```



4. Verificación del MST óptimo en el ejemplo

In [4]:
```
mst = nx.minimum_spanning_tree(G, algorithm="kruskal")

mst_edges = list(mst.edges(data=True))
mst_cost = sum(data["weight"] for _, _, data in mst_edges)

print("Aristas del MST:")
for u, v, data in mst_edges:
    print(f"{u} - {v}  peso = {data['weight']}")

print(f"\nCosto total del MST = {mst_cost}")

plt.figure(figsize=(7, 5))
nx.draw(mst, pos, with_labels=True, node_size=2000)
edge_labels_mst = {(u, v): data["weight"] for u, v, data in mst.edges(data=True)}
nx.draw_networkx_edge_labels(mst, pos, edge_labels=edge_labels_mst)
plt.title("Árbol de expansión mínima del ejemplo")
plt.axis("off")
plt.show()
```

```
Aristas del MST:
A - B  peso = 1
A - D  peso = 3
B - C  peso = 2

Costo total del MST = 6
```


Árbol de expansión mínima del ejemplo
A
B
C
D
1
2
3

Selección de Portafolios como Problema de Búsqueda

Problema: Distribuir un capital C entre n opciones de inversión, cada una con rendimiento esperado r_i.

Componente	Descripción
Espacio Ω	Símplex estándar: $\{\mathbf{x} \in \mathbb{R}^n \mid x_i \geq 0,\ \sum x_i = 1\}$
Configuración	Vector $(x_1, x_2, \ldots, x_n)$ de fracciones de capital
Tamaño	Continuo: ∞. Discretizado con paso $\delta = 1/k$: $\binom{k+n-1}{n-1}$
Función de costo	Maximizar $f(\mathbf{x}) = \sum_i x_i r_i$ s.a. $\mathrm{Var}(\mathbf{x}) \leq \sigma^2_{\max}$

In [2]:
```
# Instalación de dependencias (si es necesario)
# !pip install matplotlib networkx numpy scipy --quiet

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import networkx as nx
from itertools import product
from math import comb
from scipy.special import comb as comb_float

plt.rcParams['figure.dpi'] = 120
plt.rcParams['font.family'] = 'DejaVu Sans'
print('Librerías cargadas correctamente.')
```

```
Librerías cargadas correctamente.
```

1. Espacio de Configuraciones y su Tamaño

In [10]:
```
def tamanio_omega(n, k):
    """
    Calcula |Ω| = C(k+n-1, n-1)
    n: número de opciones de inversión
    k: número de partes iguales en que se divide el capital (1/k cada una)
    """
    return comb(k + n - 1, n - 1)

print('Tamaño del espacio de búsqueda |Ω| = C(k+n-1, n-1)\n')
print(f'{"n (opciones)":>14} | {"k=3 (δ=1/3)":>12} | {"k=5 (δ=1/5)":>12} | {"k=10 (
print('-' * 60)
for n in [2, 3, 4, 5, 10]:
    vals = [tamanio_omega(n, k) for k in [3, 5, 10]]
    print(f'{n:>14} | {vals[0]:>12,} | {vals[1]:>12,} | {vals[2]:>14,}')
```

```
Tamaño del espacio de búsqueda |Ω| = C(k+n-1, n-1)

  n (opciones) |  k=3 (δ=1/3) |  k=5 (δ=1/5) |  k=10 (δ=1/10)
-------------------------------------------------------------
             2 |            4 |            6 |             11
             3 |           10 |           21 |             66
             4 |           20 |           56 |            286
             5 |           35 |          126 |          1,001
            10 |          220 |        2,002 |         92,378
```

2. Generación del Espacio de Configuraciones

In [11]:

```python
def generar_configuraciones(n, k):
    """
    Genera todos los vectores (x_1,...,x_n) con x_i ∈ {0,1/k,...,1}
    y sum(x_i) = 1. Usa backtracking para eficiencia.
    """
    configs = []

    def backtrack(restante_k, actual, idx):
        if idx == n - 1:
            actual.append(restante_k / k)
            configs.append(tuple(actual))
            actual.pop()
            return
        for paso in range(restante_k + 1):
            actual.append(paso / k)
            backtrack(restante_k - paso, actual, idx + 1)
            actual.pop()

    backtrack(k, [], 0)
    return configs

# Ejemplo: n=3 opciones, k=3 (δ=1/3)
n, k = 3, 3
configs = generar_configuraciones(n, k)

print(f'n={n} opciones, δ=1/{k} → |Ω| = {len(configs)} configuraciones\n')
print('Todas las configuraciones (x₁, x₂, x₃):')
for i, c in enumerate(configs):
    fracs = [f'{int(x*k)}/{k}' if x not in (0.0, 1.0) else str(int(x)) for x in c]
    print(f'  {i+1:>2}. ({", ".join(fracs)})')
```

```
n=3 opciones, δ=1/3 → |Ω| = 10 configuraciones

Todas las configuraciones (x₁, x₂, x₃):
   1. (0, 0, 1)
   2. (0, 1/3, 2/3)
   3. (0, 2/3, 1/3)
   4. (0, 1, 0)
   5. (1/3, 0, 2/3)
   6. (1/3, 1/3, 1/3)
   7. (1/3, 2/3, 0)
   8. (2/3, 0, 1/3)
   9. (2/3, 1/3, 0)
  10. (1, 0, 0)
```

3. Función de Costo

In [5]:

```python
def rendimiento_esperado(x, r):
    """f(x) = sum(x_i * r_i)"""
    return np.dot(x, r)

def varianza_portafolio(x, cov):
    """Var(x) = x^T Σ x"""
    return float(np.array(x) @ cov @ np.array(x))

# Rendimientos esperados para 3 opciones
r = np.array([0.05, 0.12, 0.20])   # 5%, 12%, 20%

# Matriz de covarianza (correlación entre activos)
cov = np.array([
    [0.01,  0.002, 0.004],
    [0.002, 0.04,  0.006],
    [0.004, 0.006, 0.09 ]
])

sigma_max = 0.12   # riesgo máximo permitido

# Evaluación de todas las configuraciones
resultados = []
for c in configs:
    f = rendimiento_esperado(c, r)
    var = varianza_portafolio(c, cov)
    factible = var <= sigma_max**2
    resultados.append({'config': c, 'rendimiento': f, 'varianza': var, 'factible':

resultados.sort(key=lambda d: -d['rendimiento'])

print(f'Rendimientos r = {r}')
print(f'Riesgo máximo σ_max = {sigma_max}\n')
print(f'{"Config":>20} | {"f(x)":>8} | {"Var(x)":>8} | {"Factible":>9}')
print('-' * 55)
for d in resultados:
    c = d['config']
    fracs = [f'{int(x*k)}/{k}' if x not in (0.0,1.0) else str(int(x)) for x in c]
    cfg_str = f'({" ".join(fracs)})'
```

```
    tick = '✓' if d['factible'] else '✗'
    print(f'{cfg_str:>20} | {d["rendimiento"]:>8.4f} | {d["varianza"]:>8.4f} | {tic
```

```
Rendimientos r = [0.05 0.12 0.2 ]
Riesgo máximo σ_max = 0.12

             Config |     f(x) |   Var(x) |  Factible
-----------------------------------------------------
            (0 0 1) |   0.2000 |   0.0900 |         ✗
        (0 1/3 2/3) |   0.1733 |   0.0471 |         ✗
        (1/3 0 2/3) |   0.1500 |   0.0429 |         ✗
        (0 2/3 1/3) |   0.1467 |   0.0304 |         ✗
      (1/3 1/3 1/3) |   0.1233 |   0.0182 |         ✗
            (0 1 0) |   0.1200 |   0.0400 |         ✗
        (2/3 0 1/3) |   0.1000 |   0.0162 |         ✗
        (1/3 2/3 0) |   0.0967 |   0.0198 |         ✗
        (2/3 1/3 0) |   0.0733 |   0.0098 |         ✓
            (1 0 0) |   0.0500 |   0.0100 |         ✓
```

4. Árbol de Búsqueda (primeros 3 niveles)

In [6]:

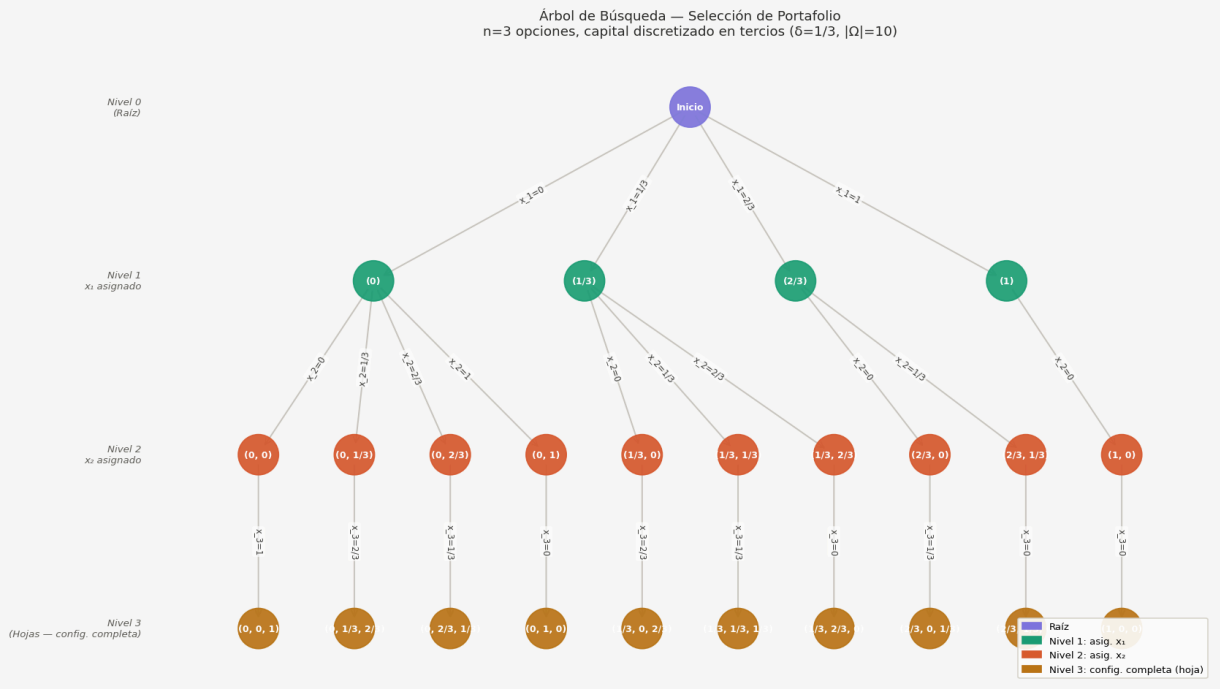
```
def construir_arbol_busqueda(n, k, max_depth=None):
    """
    Construye el árbol de búsqueda como grafo dirigido.
    Cada nodo es (asignaciones_parciales, capital_restante_en_pasos).
    """
    G = nx.DiGraph()
    root = ((), k)   # sin asignaciones, k pasos disponibles
    G.add_node(root)

    def expandir(nodo, profundidad):
        asig, restante = nodo
        nivel_actual = len(asig)
        if max_depth is not None and profundidad >= max_depth:
            return
        if nivel_actual >= n - 1:
            # Último nivel: x_n queda determinado
            hijo = (asig + (restante / k,), 0)
            G.add_node(hijo)
            G.add_edge(nodo, hijo, peso=restante / k)
            return
        for paso in range(restante + 1):
            hijo = (asig + (paso / k,), restante - paso)
            G.add_node(hijo)
            G.add_edge(nodo, hijo, peso=paso / k)
            expandir(hijo, profundidad + 1)

    expandir(root, 0)
    return G, root

G, root = construir_arbol_busqueda(n=3, k=3, max_depth=3)
print(f'Nodos en el árbol: {G.number_of_nodes()}')
print(f'Aristas en el árbol: {G.number_of_edges()}')
```

```
Nodos en el árbol: 25
Aristas en el árbol: 24
```

In [7]:
```python
def asignar_posiciones_arbol(G, root):
    """
    Calcula posiciones (x,y) para un árbol jerárquico.
    y = profundidad, x = orden entre hermanos.
    """
    pos = {}
    niveles = {}

    def bfs_niveles():
        from collections import deque
        cola = deque([(root, 0)])
        visitados = {root}
        while cola:
            nodo, nivel = cola.popleft()
            niveles.setdefault(nivel, []).append(nodo)
            for hijo in G.successors(nodo):
                if hijo not in visitados:
                    visitados.add(hijo)
                    cola.append((hijo, nivel + 1))

    bfs_niveles()
    for nivel, nodos in niveles.items():
        n_nodos = len(nodos)
        for i, nodo in enumerate(nodos):
            pos[nodo] = ((i + 1) / (n_nodos + 1), -nivel)
    return pos

pos = asignar_posiciones_arbol(G, root)

# Determinar profundidad de cada nodo
profundidad = nx.single_source_shortest_path_length(G, root)

# Colores por nivel
paleta = {
    0: '#7F77DD',   # púrpura  — raíz
    1: '#1D9E75',   # teal     — nivel 1
    2: '#D85A30',   # coral    — nivel 2
    3: '#BA7517',   # ámbar    — hojas
}
colores_nodo = [paleta.get(profundidad[n], '#888780') for n in G.nodes()]

# Etiquetas de nodos
def etiqueta(nodo):
    asig, restante = nodo
    if not asig:
        return 'Inicio'
    fracs = []
    for v in asig:
        num = int(round(v * 3))
        if num == 0: fracs.append('0')
        elif num == 3: fracs.append('1')
        else: fracs.append(f'{num}/3')
    return '(' + ', '.join(fracs) + ')'
```

```
labels = {n: etiqueta(n) for n in G.nodes()}

# Etiquetas de aristas
edge_labels = {}
for u, v, d in G.edges(data=True):
    p = d['peso']
    num = int(round(p * 3))
    nivel_u = len(u[0])
    opt = f'x_{nivel_u+1}='
    if num == 0: edge_labels[(u,v)] = opt + '0'
    elif num == 3: edge_labels[(u,v)] = opt + '1'
    else: edge_labels[(u,v)] = opt + f'{num}/3'

# ── Figura ──
fig, ax = plt.subplots(figsize=(14, 8))
ax.set_facecolor('#F8F8F6')
fig.patch.set_facecolor('#F8F8F6')

nx.draw_networkx_edges(
    G, pos, ax=ax,
    edge_color='#B4B2A9', arrows=True,
    arrowstyle='->', arrowsize=12,
    width=1.2, alpha=0.7,
    connectionstyle='arc3,rad=0.0'
)
nx.draw_networkx_nodes(
    G, pos, ax=ax,
    node_color=colores_nodo,
    node_size=1100, alpha=0.92
)
nx.draw_networkx_labels(
    G, pos, labels=labels, ax=ax,
    font_size=7.5, font_color='white', font_weight='bold'
)
nx.draw_networkx_edge_labels(
    G, pos, edge_labels=edge_labels, ax=ax,
    font_size=7, font_color='#3d3d3a',
    bbox=dict(boxstyle='round,pad=0.2', facecolor='white', alpha=0.7, edgecolor='no
)

# Anotaciones de nivel
for nivel, label in [(0,'Nivel 0\n(Raíz)'), (1,'Nivel 1\nx₁ asignado'),
                     (2,'Nivel 2\nx₂ asignado'), (3,'Nivel 3\n(Hojas — config. comp
    ax.text(-0.02, -nivel, label, ha='right', va='center',
            fontsize=8, color='#5F5E5A', style='italic')

# Leyenda
legend_patches = [
    mpatches.Patch(color='#7F77DD', label='Raíz'),
    mpatches.Patch(color='#1D9E75', label='Nivel 1: asig. x₁'),
    mpatches.Patch(color='#D85A30', label='Nivel 2: asig. x₂'),
    mpatches.Patch(color='#BA7517', label='Nivel 3: config. completa (hoja)'),
]
ax.legend(handles=legend_patches, loc='lower right', fontsize=8,
          framealpha=0.9, edgecolor='#D3D1C7')
```

```
ax.set_title('Árbol de Búsqueda — Selección de Portafolio\n'
             'n=3 opciones, capital discretizado en tercios (δ=1/3, |Ω|=10)',
             fontsize=11, pad=14, color='#2C2C2A')
ax.axis('off')
plt.tight_layout()
plt.savefig('arbol_busqueda.png', dpi=150, bbox_inches='tight')
plt.show()
print('Árbol guardado como arbol_busqueda.png')
```




```
Árbol guardado como arbol_busqueda.png
```

5. Evaluación y Mejor Configuración

In [8]:
```
# Identificar hojas (configuraciones completas)
hojas = [n for n in G.nodes() if G.out_degree(n) == 0]

print(f'Configuraciones completas (hojas): {len(hojas)}\n')

evaluaciones = []
for hoja in hojas:
    asig, _ = hoja
    x = list(asig)
    f = rendimiento_esperado(x, r)
    var = varianza_portafolio(x, cov)
    factible = var <= sigma_max**2
    evaluaciones.append((x, f, var, factible))

evaluaciones.sort(key=lambda t: -t[1])

print(f'{"Configuración":>22} | {"f(x)":>8} | {"σ(x)":>8} | {"Factible":>9}')
print('-' * 58)
for x, f, var, fact in evaluaciones:
    cfg = '(' + ', '.join([f'{int(round(v*3))}/3' if v not in (0,1) else str(int(v)
```

```
    tick = '✓' if fact else '✗'
    print(f'{cfg:>22} | {f:>8.4f} | {np.sqrt(var):>8.4f} | {tick:>9}')

mejores = [(x, f, var) for x, f, var, fact in evaluaciones if fact]
if mejores:
    x_opt, f_opt, var_opt = mejores[0]
    print(f'\n✓ Mejor configuración factible: {x_opt}')
    print(f'  Rendimiento esperado: {f_opt:.4f} ({f_opt*100:.1f}%)')
    print(f'  Desviación estándar:  {np.sqrt(var_opt):.4f} ({np.sqrt(var_opt)*100:.
```

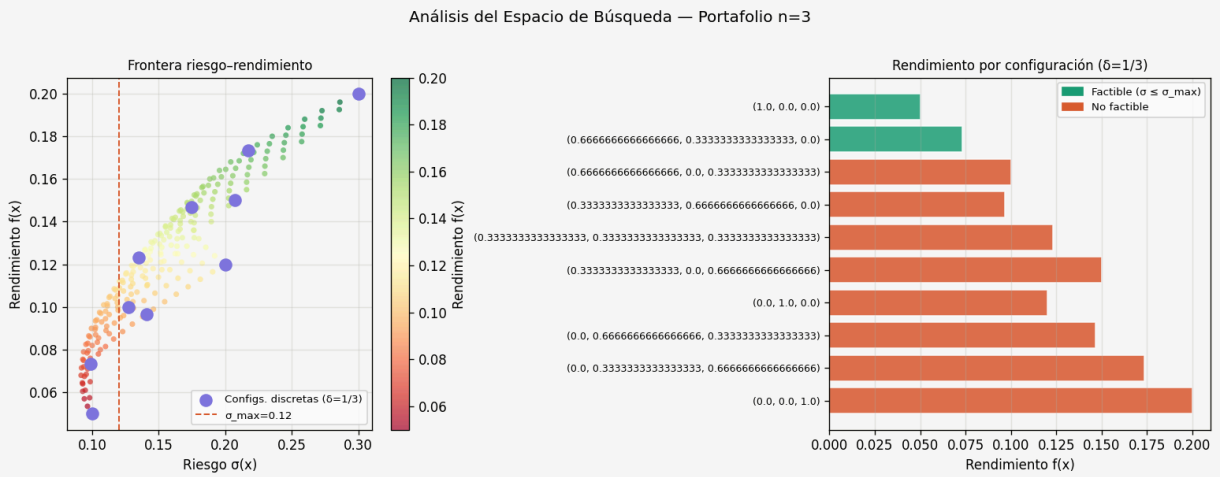
```
Configuraciones completas (hojas): 10

         Configuración |     f(x) |     σ(x) |  Factible
-------------------------------------------------------
             (0, 0, 1) |   0.2000 |   0.3000 |         ✗
         (0, 1/3, 2/3) |   0.1733 |   0.2171 |         ✗
         (1/3, 0, 2/3) |   0.1500 |   0.2071 |         ✗
         (0, 2/3, 1/3) |   0.1467 |   0.1745 |         ✗
       (1/3, 1/3, 1/3) |   0.1233 |   0.1350 |         ✗
             (0, 1, 0) |   0.1200 |   0.2000 |         ✗
         (2/3, 0, 1/3) |   0.1000 |   0.1274 |         ✗
         (1/3, 2/3, 0) |   0.0967 |   0.1406 |         ✗
         (2/3, 1/3, 0) |   0.0733 |   0.0989 |         ✓
             (1, 0, 0) |   0.0500 |   0.1000 |         ✓

✓ Mejor configuración factible: [0.6666666666666666, 0.3333333333333333, 0.0]
  Rendimiento esperado: 0.0733 (7.3%)
  Desviación estándar:  0.0989 (9.9%)
```

6. Visualización de la Frontera Eficiente (continua)

In [9]:

```
# Muestreo denso del símplex para n=3 (caso continuo aproximado)
configs_cont = generar_configuraciones(n=3, k=20)   # δ=0.05

rends = [rendimiento_esperado(c, r) for c in configs_cont]
riesgos = [np.sqrt(varianza_portafolio(c, cov)) for c in configs_cont]

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
fig.patch.set_facecolor('#F8F8F6')

# ── Panel izquierdo: riesgo vs rendimiento ──
ax1 = axes[0]
ax1.set_facecolor('#F8F8F6')
sc = ax1.scatter(riesgos, rends, c=rends, cmap='RdYlGn', s=18, alpha=0.7, linewidth
plt.colorbar(sc, ax=ax1, label='Rendimiento f(x)')

# Resaltar configuraciones discretas (k=3)
configs_disc = generar_configuraciones(n=3, k=3)
r_disc = [rendimiento_esperado(c, r) for c in configs_disc]
s_disc = [np.sqrt(varianza_portafolio(c, cov)) for c in configs_disc]
ax1.scatter(s_disc, r_disc, color='#7F77DD', s=80, zorder=5, label='Configs. discre

ax1.axvline(sigma_max, color='#D85A30', linestyle='--', lw=1.2, label=f'σ_max={sigm
ax1.set_xlabel('Riesgo σ(x)', fontsize=10)
ax1.set_ylabel('Rendimiento f(x)', fontsize=10)
```

```
ax1.set_title('Frontera riesgo-rendimiento', fontsize=10)
ax1.legend(fontsize=8)
ax1.grid(True, alpha=0.3)

# ── Panel derecho: distribución de rendimientos (barra de todas las configs discre
ax2 = axes[1]
ax2.set_facecolor('#F8F8F6')
labels_bar = [str(c) for c in configs_disc]
colores_bar = ['#1D9E75' if fact else '#D85A30'
               for _, _, _, fact in [(c, rendimiento_esperado(c,r),
                                      varianza_portafolio(c,cov),
                                      varianza_portafolio(c,cov)<=sigma_max**2)
                                     for c in configs_disc]]
ax2.barh(range(len(r_disc)), r_disc, color=colores_bar, alpha=0.85, edgecolor='whit
ax2.set_yticks(range(len(r_disc)))
ax2.set_yticklabels([str(c) for c in configs_disc], fontsize=7.5)
ax2.set_xlabel('Rendimiento f(x)', fontsize=10)
ax2.set_title('Rendimiento por configuración (δ=1/3)', fontsize=10)
ax2.axvline(0, color='gray', lw=0.5)
legend_elems = [
    mpatches.Patch(color='#1D9E75', label='Factible (σ ≤ σ_max)'),
    mpatches.Patch(color='#D85A30', label='No factible'),
]
ax2.legend(handles=legend_elems, fontsize=8)
ax2.grid(True, axis='x', alpha=0.3)

plt.suptitle('Análisis del Espacio de Búsqueda — Portafolio n=3', fontsize=12, y=1.
plt.tight_layout()
plt.savefig('frontera_eficiente.png', dpi=150, bbox_inches='tight')
plt.show()
print('Gráfica guardada como frontera_eficiente.png')
```




```
Gráfica guardada como frontera_eficiente.png
```

Problema 3

TSP como problema de búsqueda

1. Formulación como problema de búsqueda

En el problema del viajero se tiene un conjunto de ciudades y una distancia entre cada par de ellas.

El objetivo es:

1. iniciar en una ciudad,
2. visitar cada ciudad una sola vez,
3. regresar a la ciudad de inicio,
4. y lograr que la distancia total sea la menor posible.

Espacio de configuraciones

Las posibles soluciones son todas las rutas que empiezan en una ciudad, pasan por todas las demás una sola vez y al final regresan al punto de inicio.

En otras palabras, cada configuración representa un posible recorrido completo entre las ciudades.

También pueden existir configuraciones parciales, es decir, rutas que todavía no incluyen todas las ciudades, pero que ya muestran parte del recorrido.

Tamaño del espacio de búsqueda

La cantidad de rutas posibles depende del número de ciudades.

Mientras más ciudades haya, más grande será el número de posibles recorridos.

Descripción de las configuraciones

Cada configuración puede representarse como un orden en el que se visitan las ciudades.

Durante la búsqueda, también se puede trabajar con rutas parciales, agregando una ciudad nueva en cada paso sin repetir las anteriores.

Función de costo

El costo de una solución se obtiene sumando todas las distancias recorridas, incluyendo el regreso a la ciudad inicial.

Objetivo

El objetivo es encontrar la ruta con la menor distancia total posible.

Componentes de búsqueda

- **Estado inicial:** ciudad de inicio.
- **Acción:** agregar una ciudad que no haya sido visitada.
- **Restricción:** no repetir ciudades.
- **Meta:** visitar todas las ciudades y regresar al punto de partida.

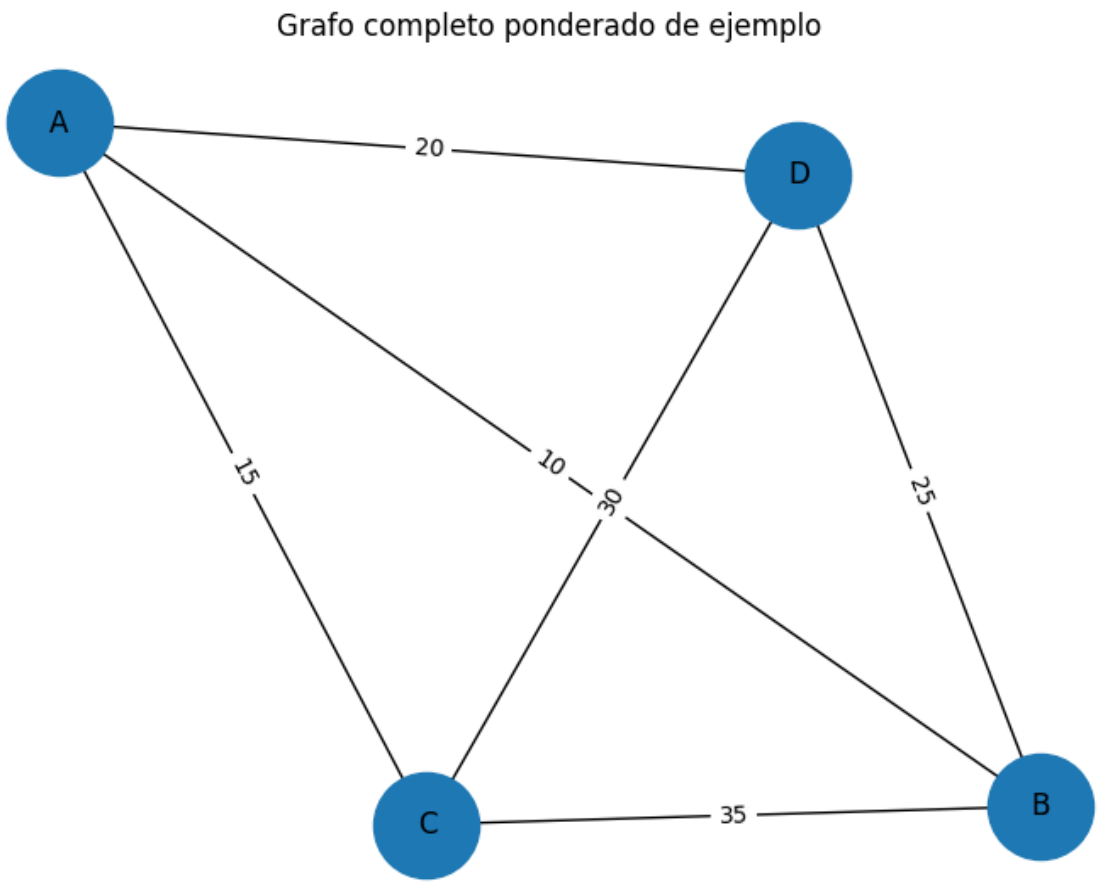
In [1]:
```
import itertools
import networkx as nx
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (8, 5)
```

2. Grafo completo pequeño de ejemplo

Usaremos 4 ciudades para que el árbol de búsqueda se pueda visualizar fácilmente.

In [2]:
```
cities = ["A", "B", "C", "D"]

weights = {
    ("A", "B"): 10,
    ("A", "C"): 15,
    ("A", "D"): 20,
    ("B", "C"): 35,
    ("B", "D"): 25,
    ("C", "D"): 30,
}

def dist(u, v):
    if u == v:
        return 0
    key = tuple(sorted((u, v)))
    return weights[key]

G = nx.Graph()
for (u, v), w in weights.items():
    G.add_edge(u, v, weight=w)

pos = nx.spring_layout(G, seed=9)

plt.figure(figsize=(7, 5))
nx.draw(G, pos, with_labels=True, node_size=2000)
nx.draw_networkx_edge_labels(G, pos, edge_labels=weights)
```

```
plt.title("Grafo completo ponderado de ejemplo")
plt.axis("off")
plt.show()
```



3. Árbol de búsqueda

Se fijará la ciudad inicial como **A**.

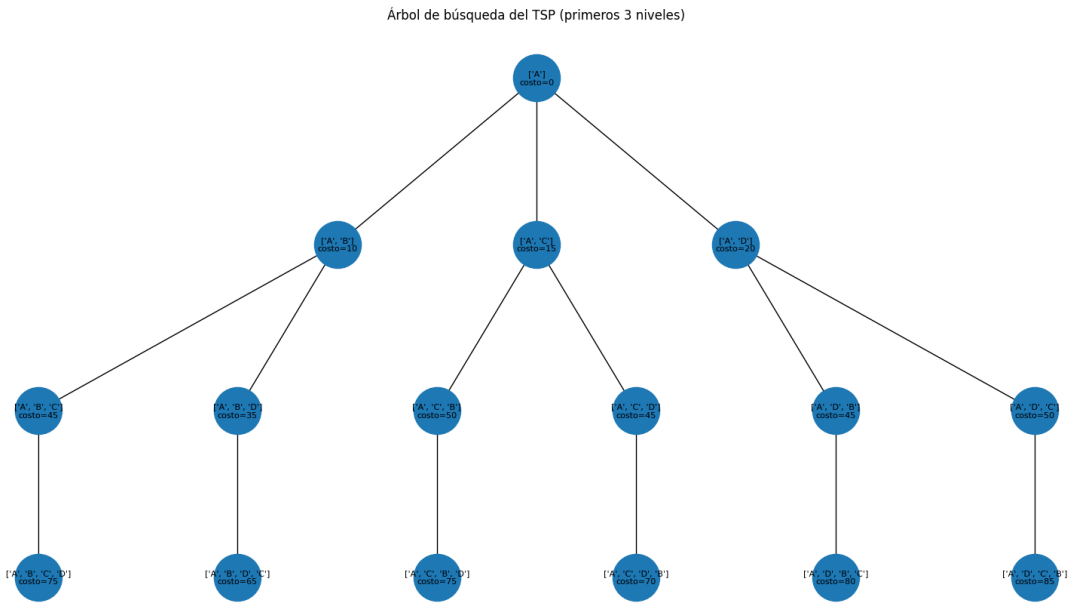
Entonces:

- la raíz es `['A']`,
- en el nivel 1 se elige la segunda ciudad,
- en el nivel 2 se elige la tercera ciudad,
- en el nivel 3 se elige la cuarta ciudad.

Después de eso, solo faltaría regresar a `A` para cerrar el ciclo.

In [3]:
```
def partial_cost(route):
    return sum(dist(route[i], route[i + 1]) for i in range(len(route) - 1))

def build_tsp_search_tree(cities, start="A", max_depth=3):
    T = nx.DiGraph()
    root = (start,)
    T.add_node(root, level=0, label=f"{list(root)}\ncosto=0")
```

```
    def expand(route):
        level = len(route) - 1
        if level == max_depth:
            return

        remaining = [c for c in cities if c not in route]
        for c in remaining:
            new_route = tuple(list(route) + [c])
            T.add_node(
                new_route,
                level=len(new_route) - 1,
                label=f"{list(new_route)}\ncosto={partial_cost(new_route)}",
            )
            T.add_edge(route, new_route)
            expand(new_route)

    expand(root)
    return T

search_tree = build_tsp_search_tree(cities, start="A", max_depth=3)

def layered_positions(G):
    levels = {}
    for node, data in G.nodes(data=True):
        lvl = data["level"]
        levels.setdefault(lvl, []).append(node)

    pos = {}
    for lvl, nodes in levels.items():
        width = len(nodes)
        for i, node in enumerate(nodes):
            x = i - (width - 1) / 2
            y = -lvl
            pos[node] = (x, y)
    return pos

pos_tree = layered_positions(search_tree)
labels_tree = nx.get_node_attributes(search_tree, "label")

plt.figure(figsize=(16, 8))
nx.draw(search_tree, pos_tree, with_labels=False, node_size=2000, arrows=False)
nx.draw_networkx_labels(search_tree, pos_tree, labels=labels_tree, font_size=8)
plt.title("Árbol de búsqueda del TSP (primeros 3 niveles)")
plt.axis("off")
plt.show()
```



4. Búsqueda por fuerza bruta en el ejemplo

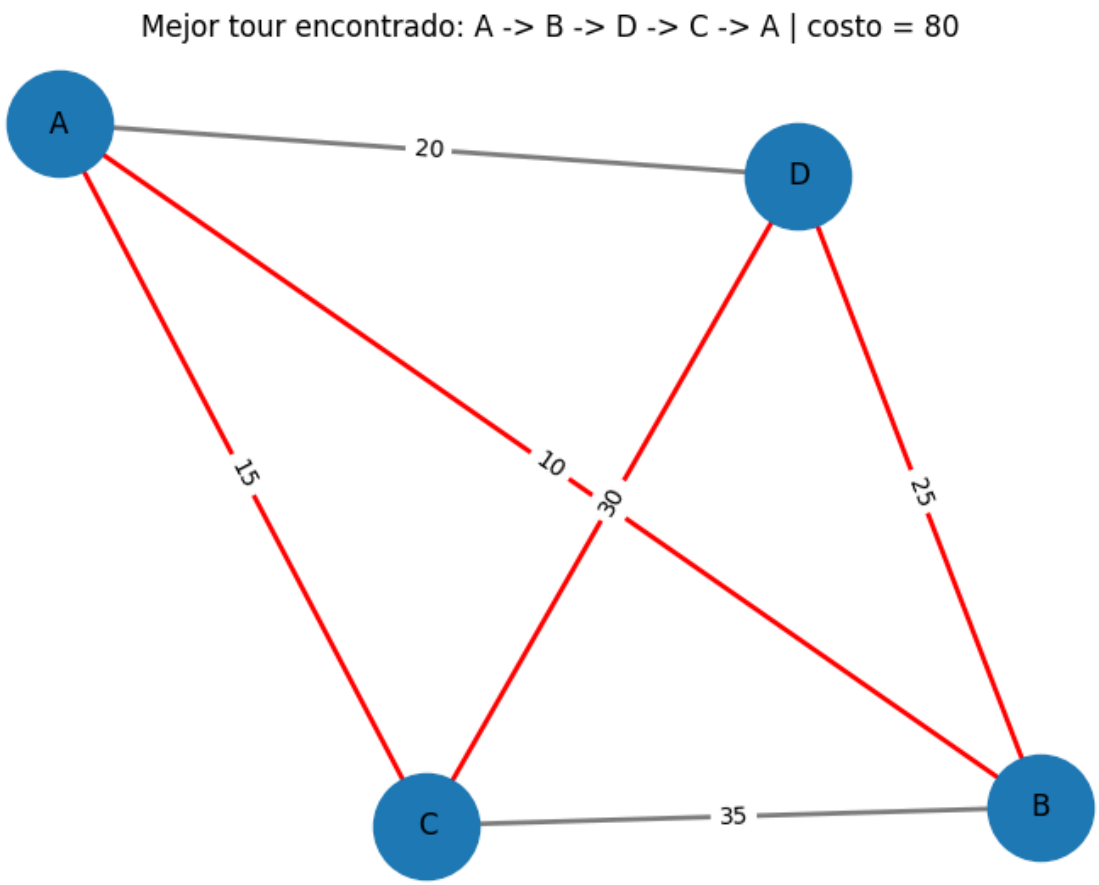
Para un número pequeño de ciudades se puede revisar todas las rutas posibles y escoger la de menor costo.

In [4]:
```
start = "A"
other_cities = [c for c in cities if c != start]

best_route = None
best_cost = float("inf")

print("Rutas posibles desde A:\n")
for perm in itertools.permutations(other_cities):
    route = (start,) + perm + (start,)
    cost = sum(dist(route[i], route[i + 1]) for i in range(len(route) - 1))
    print(route, "-> costo =", cost)

    if cost < best_cost:
        best_cost = cost
        best_route = route

print("\nMejor ruta encontrada:")
print(best_route)
print("Costo mínimo =", best_cost)
```

```
Rutas posibles desde A:

('A', 'B', 'C', 'D', 'A') -> costo = 95
('A', 'B', 'D', 'C', 'A') -> costo = 80
('A', 'C', 'B', 'D', 'A') -> costo = 95
('A', 'C', 'D', 'B', 'A') -> costo = 80
('A', 'D', 'B', 'C', 'A') -> costo = 95
('A', 'D', 'C', 'B', 'A') -> costo = 95

Mejor ruta encontrada:
('A', 'B', 'D', 'C', 'A')
Costo mínimo = 80
```

In [5]:

```
tour_edges = [(best_route[i], best_route[i + 1]) for i in range(len(best_route) - 1
edge_colors = ["red" if (u, v) in tour_edges or (v, u) in tour_edges else "gray" fo

plt.figure(figsize=(7, 5))
nx.draw(G, pos, with_labels=True, node_size=2000, edge_color=edge_colors, width=2)
nx.draw_networkx_edge_labels(G, pos, edge_labels=weights)
plt.title(f"Mejor tour encontrado: {' -> '.join(best_route)} | costo = {best_cost}"
plt.axis("off")
plt.show()
```



Sudoku 9×9 como Problema de Búsqueda

Problema: Rellenar una cuadrícula 9×9 de modo que cada fila, columna y subcuadro 3×3 contenga los dígitos 1–9 exactamente una vez.

Componente	Descripción
Espacio Ω	Todas las asignaciones de dígitos 1–9 a las 81 celdas
Configuración	Matriz 9×9 con entradas en $\{1, \ldots, 9\}$
Tamaño	$9^{81} \approx 1.97 \times 10^{77}$ sin restricciones; $\approx 6.67 \times 10^{21}$ soluciones válidas
Función de costo	Número de conflictos (repeticiones en filas, columnas y cajas) = 0 en solución
Estrategia	Backtracking + Forward Checking (MRV)

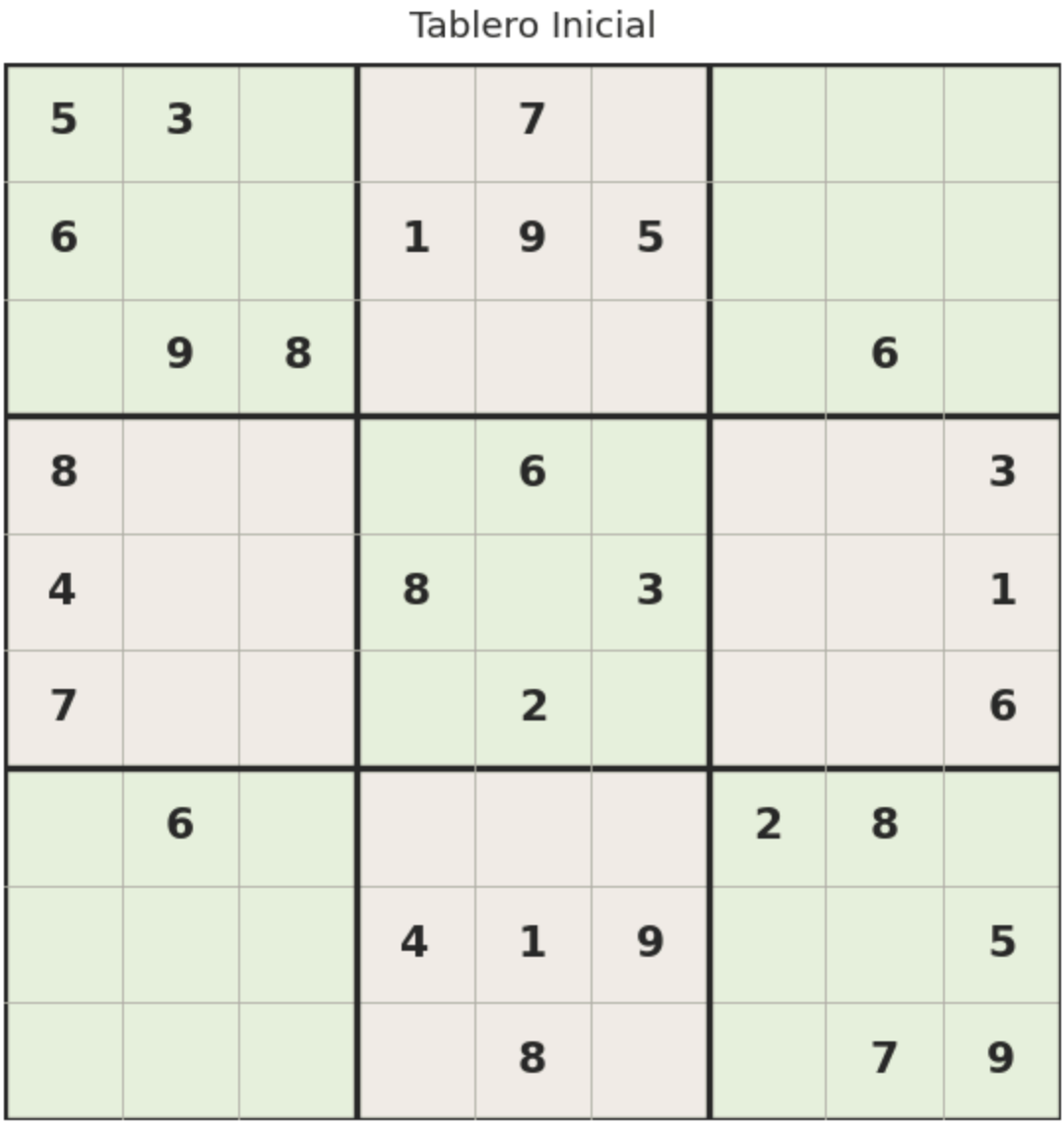
In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from copy import deepcopy

plt.rcParams['figure.dpi'] = 120

# ── Tablero inicial ──
# 0 = celda vacía
tablero_inicial = [
    [5, 3, 0, 0, 7, 0, 0, 0, 0],
    [6, 0, 0, 1, 9, 5, 0, 0, 0],
    [0, 9, 8, 0, 0, 0, 0, 6, 0],
    [8, 0, 0, 0, 6, 0, 0, 0, 3],
    [4, 0, 0, 8, 0, 3, 0, 0, 1],
    [7, 0, 0, 0, 2, 0, 0, 0, 6],
    [0, 6, 0, 0, 0, 0, 2, 8, 0],
    [0, 0, 0, 4, 1, 9, 0, 0, 5],
    [0, 0, 0, 0, 8, 0, 0, 7, 9],
]

print('Tablero inicial (0 = vacío):')
for fila in tablero_inicial:
    print(fila)
```

```
Tablero inicial (0 = vacío):
[5, 3, 0, 0, 7, 0, 0, 0, 0]
[6, 0, 0, 1, 9, 5, 0, 0, 0]
[0, 9, 8, 0, 0, 0, 0, 6, 0]
[8, 0, 0, 0, 6, 0, 0, 0, 3]
[4, 0, 0, 8, 0, 3, 0, 0, 1]
[7, 0, 0, 0, 2, 0, 0, 0, 6]
[0, 6, 0, 0, 0, 0, 2, 8, 0]
[0, 0, 0, 4, 1, 9, 0, 0, 5]
[0, 0, 0, 0, 8, 0, 0, 7, 9]
```

1. Visualización del tablero inicial

In [2]:
```
def dibujar_sudoku(tablero, titulo='Sudoku', dado=None, ax=None):
    """
    Dibuja el tablero de Sudoku.
    dado: conjunto de (fila, col) con celdas originales (se pintan diferente).
    """
    if ax is None:
        fig, ax = plt.subplots(figsize=(5, 5))
        standalone = True
    else:
        standalone = False

    ax.set_xlim(0, 9)
    ax.set_ylim(0, 9)
    ax.set_aspect('equal')
    ax.axis('off')
    ax.set_title(titulo, fontsize=11, pad=8, color='#2C2C2A')

    # Fondo de subcajas 3×3
    for bi in range(3):
        for bj in range(3):
            color = '#EAF3DE' if (bi + bj) % 2 == 0 else '#F1EFE8'
            rect = patches.Rectangle((bj*3, (2-bi)*3), 3, 3,
                                     facecolor=color, edgecolor='none')
            ax.add_patch(rect)

    # Líneas finas (celdas)
    for i in range(10):
        lw = 2.0 if i % 3 == 0 else 0.5
        color = '#2C2C2A' if i % 3 == 0 else '#B4B2A9'
        ax.axhline(i, 0, 9, color=color, lw=lw)
        ax.axvline(i, 0, 9, color=color, lw=lw)

    # Números
    for i in range(9):
        for j in range(9):
            val = tablero[i][j]
            if val != 0:
                es_dado = dado is None or (i, j) in dado
                color_txt = '#2C2C2A' if es_dado else '#185FA5'
                weight = 'bold' if es_dado else 'normal'
                ax.text(j + 0.5, 8.5 - i, str(val),
                        ha='center', va='center',
                        fontsize=13, color=color_txt,
                        fontweight=weight)

    if standalone:
        plt.tight_layout()
        plt.show()

# Identificar celdas dadas
celdas_dadas = {(i, j) for i in range(9) for j in range(9)
                if tablero_inicial[i][j] != 0}
```

```
dibujar_sudoku(tablero_inicial, titulo='Tablero Inicial', dado=celdas_dadas)
```



2. Espacio de Configuraciones y Función de Costo

In [3]:
```
vacias = sum(tablero_inicial[i][j] == 0
             for i in range(9) for j in range(9))

print('=== Espacio de configuraciones ===')
print(f'Celdas totales:        81')
print(f'Celdas dadas:          {81 - vacias}')
print(f'Celdas vacías:         {vacias}')
print(f'|Ω| sin restricciones: 9^{vacias} ≈ {9**vacias:.3e}')
print()

def contar_conflictos(tablero):
    """Función de costo: número de conflictos (repeticiones)."""
    conflictos = 0
    for i in range(9):
```

```
        fila = [tablero[i][j] for j in range(9) if tablero[i][j] != 0]
        conflictos += len(fila) - len(set(fila))
        col = [tablero[j][i] for j in range(9) if tablero[j][i] != 0]
        conflictos += len(col) - len(set(col))
    for bi in range(3):
        for bj in range(3):
            caja = [tablero[bi*3+di][bj*3+dj]
                    for di in range(3) for dj in range(3)
                    if tablero[bi*3+di][bj*3+dj] != 0]
            conflictos += len(caja) - len(set(caja))
    return conflictos

print(f'Conflictos tablero inicial: {contar_conflictos(tablero_inicial)}')
print()
print('Función de costo: f(tablero) = número de conflictos')
print('Objetivo: encontrar tablero con f = 0')
```

```
=== Espacio de configuraciones ===
Celdas totales:        81
Celdas dadas:          30
Celdas vacías:         51
|Ω| sin restricciones: 9^51 ≈ 4.638e+48

Conflictos tablero inicial: 0

Función de costo: f(tablero) = número de conflictos
Objetivo: encontrar tablero con f = 0
```

3. Árbol de Búsqueda — Primeros Niveles

Cada nivel asigna un valor a una celda vacía. El **factor de ramificación** varía:

- Sin restricciones: hasta 9 hijos por nodo.
- Con Forward Checking (MRV): se elige primero la celda con menos valores posibles.

In [4]:
```
def valores_posibles(tablero, fila, col):
    """Devuelve el conjunto de valores válidos para la celda (fila, col)."""
    usados = set()
    usados.update(tablero[fila])                          # fila
    usados.update(tablero[r][col] for r in range(9))      # columna
    bi, bj = (fila // 3) * 3, (col // 3) * 3
    usados.update(tablero[bi+di][bj+dj]                   # caja 3×3
                  for di in range(3) for dj in range(3))
    return set(range(1, 10)) - usados

def mrv(tablero):
    """Minimum Remaining Values: celda vacía con menos opciones."""
    mejor, min_ops = None, 10
    for i in range(9):
        for j in range(9):
            if tablero[i][j] == 0:
                ops = len(valores_posibles(tablero, i, j))
                if ops < min_ops:
                    mejor, min_ops = (i, j), ops
```

```
    return mejor

# Mostrar los primeros 3 niveles del árbol
print('Árbol de búsqueda — primeros 3 niveles (con MRV)\n')
print('Nivel 0 (Raíz): tablero inicial')

c0 = mrv(tablero_inicial)
ops0 = sorted(valores_posibles(tablero_inicial, *c0))
print(f'\nNivel 1 → celda MRV: {c0}, valores posibles: {ops0}')

for v0 in ops0[:3]:   # sólo los 3 primeros hijos para no saturar la salida
    t1 = deepcopy(tablero_inicial)
    t1[c0[0]][c0[1]] = v0
    c1 = mrv(t1)
    if c1:
        ops1 = sorted(valores_posibles(t1, *c1))
        print(f'  ├─ celda {c0}={v0}  →  Nivel 2: celda MRV {c1}, opciones {ops1}')
        for v1 in ops1[:2]:
            t2 = deepcopy(t1)
            t2[c1[0]][c1[1]] = v1
            c2 = mrv(t2)
            if c2:
                ops2 = sorted(valores_posibles(t2, *c2))
                print(f'  │   ├─ celda {c1}={v1}  →  Nivel 3: celda MRV {c2}, opcio

print('\n  (ramas podadas cuando valores_posibles = ∅ → backtrack)')
```

```
Árbol de búsqueda — primeros 3 niveles (con MRV)

Nivel 0 (Raíz): tablero inicial

Nivel 1 → celda MRV: (4, 4), valores posibles: [5]
  ├─ celda (4, 4)=5  →  Nivel 2: celda MRV (4, 1), opciones [2]
  │   ├─ celda (4, 1)=2  →  Nivel 3: celda MRV (4, 7), opciones [9]

  (ramas podadas cuando valores_posibles = ∅ → backtrack)
```

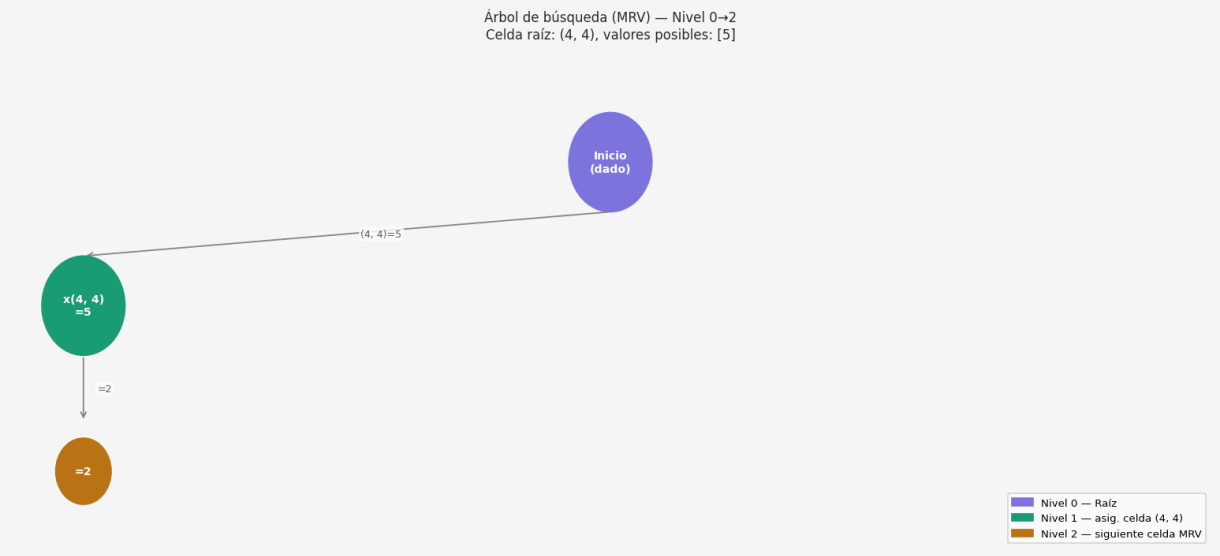
4. Visualización del árbol (grafo primeros 2 niveles)

In [5]:

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

fig, ax = plt.subplots(figsize=(13, 6))
ax.set_facecolor('#F8F8F6')
fig.patch.set_facecolor('#F8F8F6')
ax.axis('off')
ax.set_xlim(0, 13)
ax.set_ylim(-3.5, 1)

PALETA = {
    0: ('#7F77DD', 'white'),   # raíz — púrpura
    1: ('#1D9E75', 'white'),   # nivel 1 — teal
    2: ('#BA7517', 'white'),   # nivel 2 — ámbar
}
```

```
def dibujar_nodo(ax, cx, cy, texto, nivel, radio=0.45):
    color, tcolor = PALETA[nivel]
    circ = plt.Circle((cx, cy), radio, color=color, zorder=3)
    ax.add_patch(circ)
    ax.text(cx, cy, texto, ha='center', va='center',
            fontsize=8.5, color=tcolor, fontweight='bold', zorder=4)

def dibujar_arista(ax, x1, y1, x2, y2, etiqueta=''):
    ax.annotate('', xy=(x2, y2+0.45), xytext=(x1, y1-0.45),
                arrowprops=dict(arrowstyle='->', color='#888780', lw=1.1))
    mx, my = (x1+x2)/2, (y1+y2)/2
    if etiqueta:
        ax.text(mx+0.15, my, etiqueta, ha='left', va='center',
                fontsize=7.5, color='#5F5E5A',
                bbox=dict(boxstyle='round,pad=0.2', fc='white', ec='none', alpha=0.

# Raíz
dibujar_nodo(ax, 6.5, 0, 'Inicio\n(dado)', 0)

# Nivel 1
c0 = mrv(tablero_inicial)
ops0 = sorted(valores_posibles(tablero_inicial, *c0))
n1 = len(ops0)
xs1 = np.linspace(0.8, 12.2, n1)

for idx, v0 in enumerate(ops0):
    x1 = xs1[idx]
    dibujar_arista(ax, 6.5, 0, x1, -1.3, f'{c0}={v0}')
    t1 = deepcopy(tablero_inicial)
    t1[c0[0]][c0[1]] = v0
    c1 = mrv(t1)
    ops1 = sorted(valores_posibles(t1, *c1)) if c1 else []
    n2 = len(ops1)
    label_n1 = f'x{c0}\n={v0}'
    dibujar_nodo(ax, x1, -1.3, label_n1, 1)

    # Nivel 2 (solo primeros 2 hijos por nodo nivel-1)
    if ops1 and idx < 5:   # limitar para no saturar
        sub = ops1[:2]
        spread = 0.55
        xs2 = [x1 + (i - (len(sub)-1)/2) * spread for i in range(len(sub))]
        for jdx, v1 in enumerate(sub):
            x2 = xs2[jdx]
            dibujar_arista(ax, x1, -1.3, x2, -2.8, f'={v1}')
            dibujar_nodo(ax, x2, -2.8, f'={v1}', 2, radio=0.3)
        if len(ops1) > 2:
            ax.text(x1 + 0.35, -2.8, f'+{len(ops1)-2}…',
                    va='center', fontsize=7, color='#888780')

# Leyenda
legend_items = [
    mpatches.Patch(color='#7F77DD', label='Nivel 0 — Raíz'),
    mpatches.Patch(color='#1D9E75', label=f'Nivel 1 — asig. celda {c0}'),
    mpatches.Patch(color='#BA7517', label='Nivel 2 — siguiente celda MRV'),
]
ax.legend(handles=legend_items, loc='lower right', fontsize=8,
```

```
          framealpha=0.9, edgecolor='#D3D1C7')

ax.set_title(f'Árbol de búsqueda (MRV) — Nivel 0→2\n'
             f'Celda raíz: {c0}, valores posibles: {ops0}',
             fontsize=10, pad=10, color='#2C2C2A')

plt.tight_layout()
plt.savefig('arbol_sudoku.png', dpi=150, bbox_inches='tight')
plt.show()
```



5. Solucionador — Backtracking + MRV

In [6]:
```
stats = {'nodos': 0, 'podas': 0}

def resolver(tablero):
    """
    Backtracking con MRV (Minimum Remaining Values).
    Retorna True si encontró solución, modificando tablero in-place.
    """
    celda = mrv(tablero)
    if celda is None:
        return True   # todas las celdas asignadas → solución

    fila, col = celda
    for val in sorted(valores_posibles(tablero, fila, col)):
        stats['nodos'] += 1
        tablero[fila][col] = val
        if resolver(tablero):
            return True
        tablero[fila][col] = 0   # backtrack
        stats['podas'] += 1

    return False

solucion = deepcopy(tablero_inicial)
exito = resolver(solucion)
```

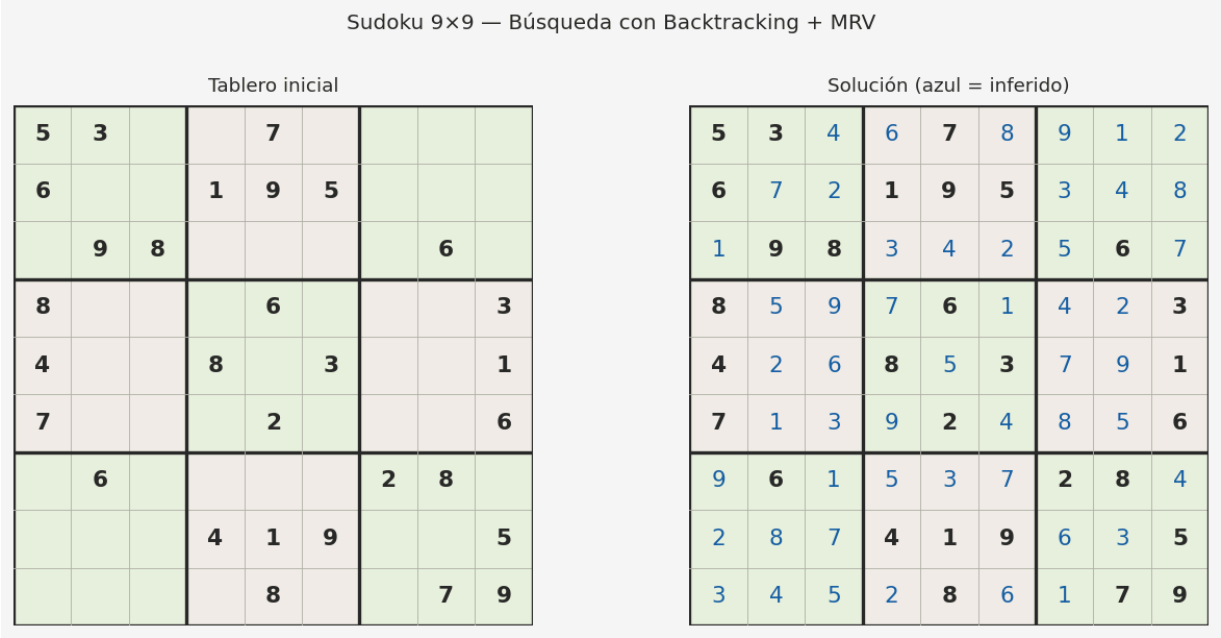
```
print(f'Solución encontrada: {exito}')
print(f'Nodos explorados:    {stats["nodos"]}')
print(f'Backtracks:          {stats["podas"]}')
print(f'Conflictos finales:  {contar_conflictos(solucion)}')
```

```
Solución encontrada: True
Nodos explorados:    51
Backtracks:          0
Conflictos finales:  0
```

6. Visualización de la solución

In [7]:

```
fig, axes = plt.subplots(1, 2, figsize=(11, 5))
fig.patch.set_facecolor('#F8F8F6')

dibujar_sudoku(tablero_inicial, titulo='Tablero inicial',
               dado=celdas_dadas, ax=axes[0])
dibujar_sudoku(solucion, titulo='Solución (azul = inferido)',
               dado=celdas_dadas, ax=axes[1])

plt.suptitle('Sudoku 9×9 — Búsqueda con Backtracking + MRV',
             fontsize=12, y=1.01, color='#2C2C2A')
plt.tight_layout()
plt.savefig('sudoku_solucion.png', dpi=150, bbox_inches='tight')
plt.show()

print('\nSolución completa:')
for fila in solucion:
    print(fila)
```



```
Solución completa:
[5, 3, 4, 6, 7, 8, 9, 1, 2]
[6, 7, 2, 1, 9, 5, 3, 4, 8]
[1, 9, 8, 3, 4, 2, 5, 6, 7]
[8, 5, 9, 7, 6, 1, 4, 2, 3]
[4, 2, 6, 8, 5, 3, 7, 9, 1]
[7, 1, 3, 9, 2, 4, 8, 5, 6]
[9, 6, 1, 5, 3, 7, 2, 8, 4]
[2, 8, 7, 4, 1, 9, 6, 3, 5]
[3, 4, 5, 2, 8, 6, 1, 7, 9]
```

Problema 5 - Analisis de Redes Sociales

Librerias

In [2]:
```
import networkx as nx
import matplotlib.pyplot as plt
from networkx.algorithms.community import greedy_modularity_communities

plt.rcParams["figure.figsize"] = (8, 6)
```

(a) Cargar el grafo de red social

In [3]:
```
# Cargar grafo de red social predefinido
G = nx.karate_club_graph()

print("Grafo cargado correctamente.")
```

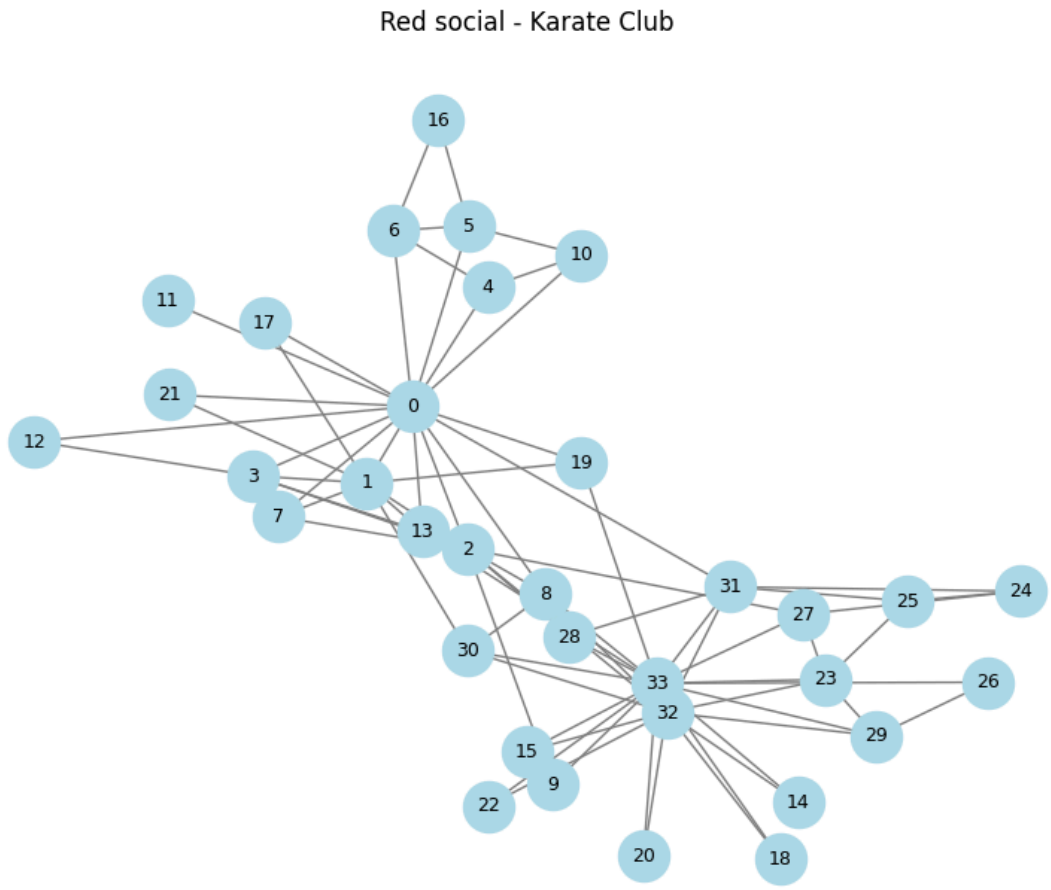
```
Grafo cargado correctamente.
```

(b) Obtener informacion del grafo

In [4]:
```
num_nodos = G.number_of_nodes()
num_aristas = G.number_of_edges()

print("Numero de nodos:", num_nodos)
print("Numero de aristas:", num_aristas)
```

```
Numero de nodos: 34
Numero de aristas: 78
```

In [5]:
```
# Visualizacion basica del grafo
pos = nx.spring_layout(G, seed=42)

plt.figure()
nx.draw(
    G,
    pos,
    with_labels=True,
    node_color="lightblue",
    edge_color="gray",
    node_size=600,
    font_size=9
)
plt.title("Red social - Karate Club")
plt.show()
```



(c) Tres formas distintas de definir un "influencer"

1. Un nodo se le considera influencer si tiene muchas conexiones directas.

2. Un nodo es influencer si sirve de puente entre distintos grupos de la red.

3. Un nodo es influencer si esta conectado con otros nodos importantes.

In [6]:
```
# Calcular centralidades
degree_centrality = nx.degree_centrality(G)
betweenness_centrality = nx.betweenness_centrality(G)
eigenvector_centrality = nx.eigenvector_centrality(G, max_iter=1000)

print("Centralidades calculadas.")
```

```
Centralidades calculadas.
```

In [7]:
```
# Mostrar los 5 nodos mas altos en cada metrica

top5_degree = sorted(degree_centrality.items(), key=lambda x: x[1], reverse=True)[:
```

```
top5_betweenness = sorted(betweenness_centrality.items(), key=lambda x: x[1], rever
top5_eigenvector = sorted(eigenvector_centrality.items(), key=lambda x: x[1], rever

print("Top 5 - Degree Centrality")
for nodo, valor in top5_degree:
    print(f"Nodo {nodo}: {valor:.4f}")

print("\nTop 5 - Betweenness Centrality")
for nodo, valor in top5_betweenness:
    print(f"Nodo {nodo}: {valor:.4f}")

print("\nTop 5 - Eigenvector Centrality")
for nodo, valor in top5_eigenvector:
    print(f"Nodo {nodo}: {valor:.4f}")
```

```
Top 5 - Degree Centrality
Nodo 33: 0.5152
Nodo 0: 0.4848
Nodo 32: 0.3636
Nodo 2: 0.3030
Nodo 1: 0.2727

Top 5 - Betweenness Centrality
Nodo 0: 0.4376
Nodo 33: 0.3041
Nodo 32: 0.1452
Nodo 2: 0.1437
Nodo 31: 0.1383

Top 5 - Eigenvector Centrality
Nodo 33: 0.3734
Nodo 0: 0.3555
Nodo 2: 0.3172
Nodo 32: 0.3087
Nodo 1: 0.2660
```

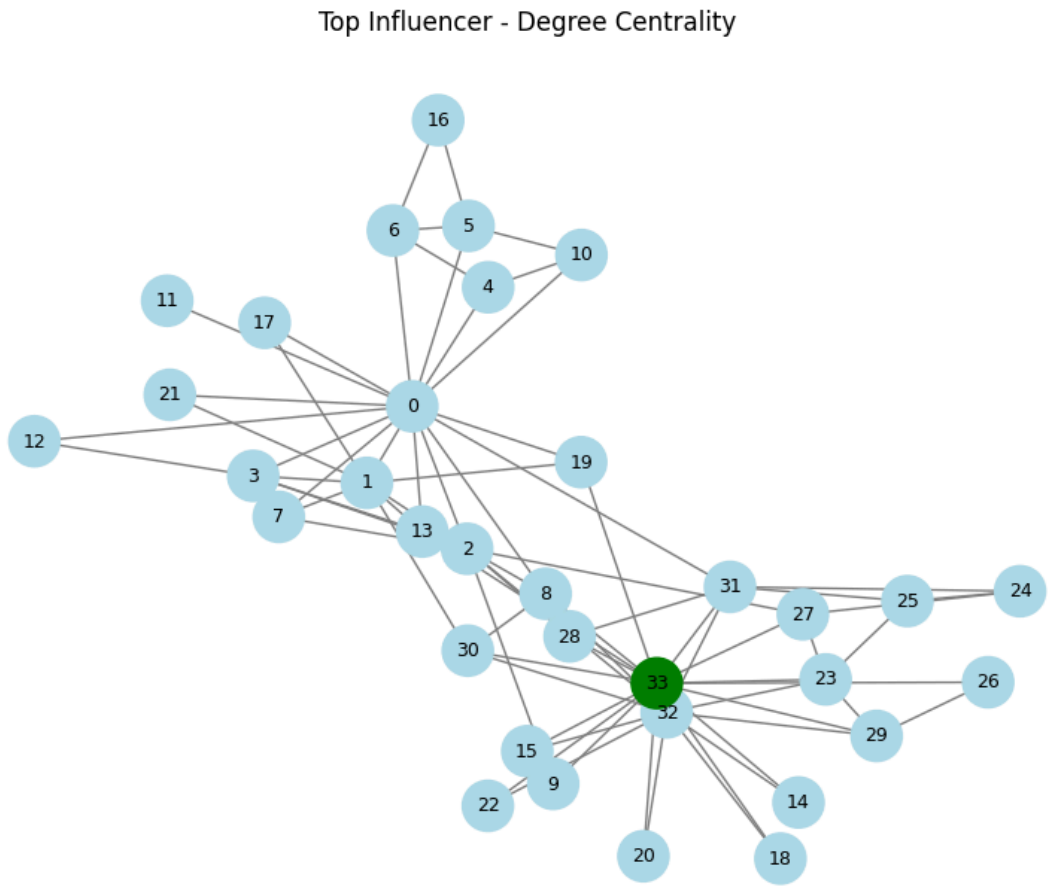
(d) Encontrar el Top Influencer segun cada propuesta

In [8]:
```
top_degree = max(degree_centrality, key=degree_centrality.get)
top_betweenness = max(betweenness_centrality, key=betweenness_centrality.get)
top_eigenvector = max(eigenvector_centrality, key=eigenvector_centrality.get)

print("Top influencer segun Degree Centrality:", top_degree)
print("Top influencer segun Betweenness Centrality:", top_betweenness)
print("Top influencer segun Eigenvector Centrality:", top_eigenvector)
```

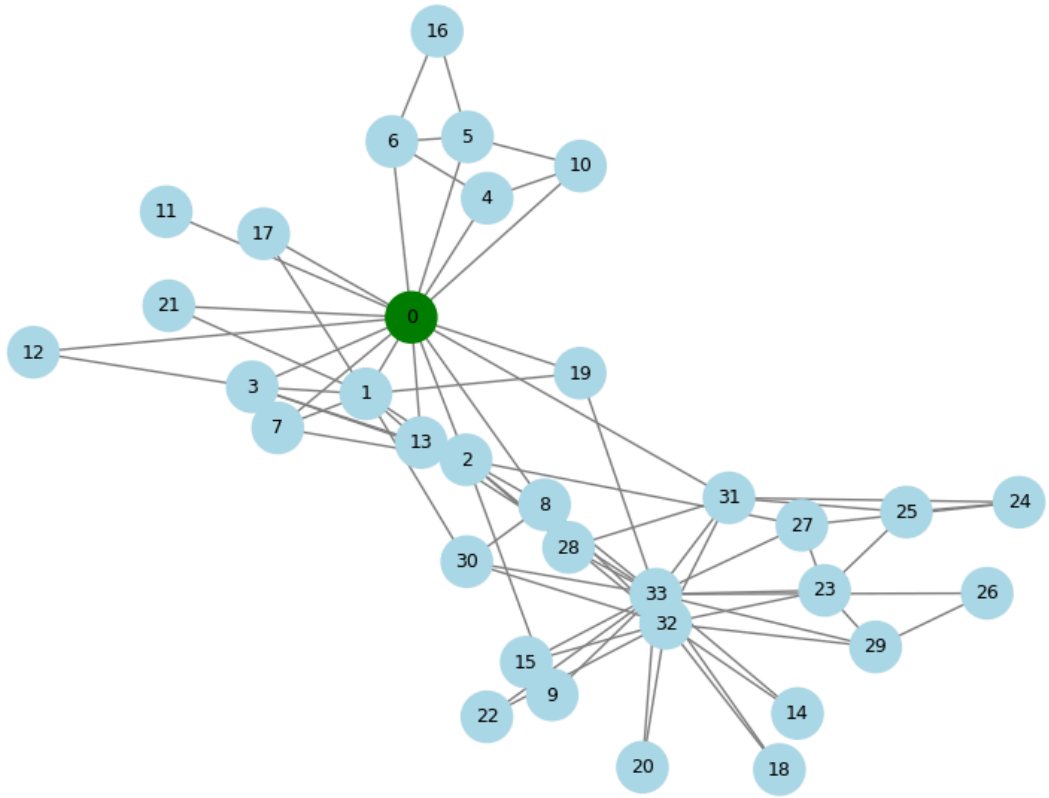
```
Top influencer segun Degree Centrality: 33
Top influencer segun Betweenness Centrality: 0
Top influencer segun Eigenvector Centrality: 33
```

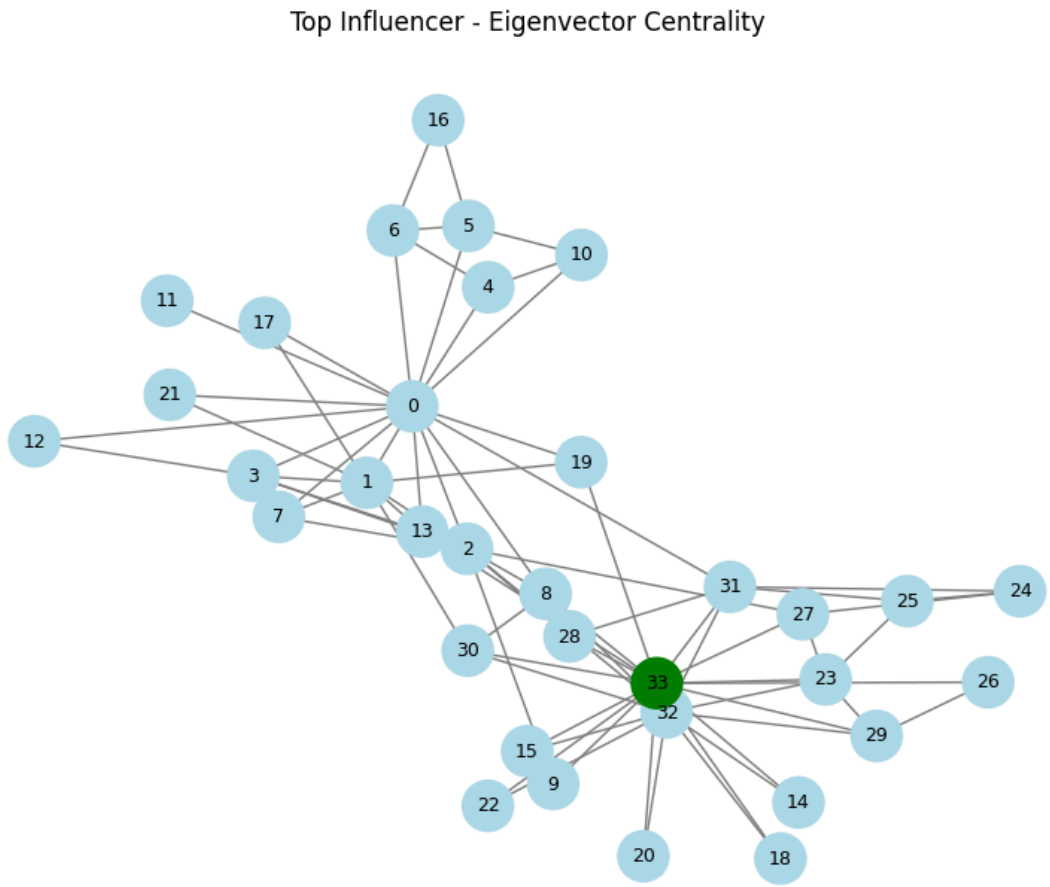
In [9]:
```
def dibujar_influencer(grafo, pos, influencer, titulo):
    colores = ["green" if n == influencer else "lightblue" for n in grafo.nodes()]
    plt.figure()
    nx.draw(
```

```
        grafo,
        pos,
        with_labels=True,
        node_color=colores,
        edge_color="gray",
        node_size=600,
        font_size=9
    )
    plt.title(titulo)
    plt.show()

dibujar_influencer(G, pos, top_degree, "Top Influencer - Degree Centrality")
dibujar_influencer(G, pos, top_betweenness, "Top Influencer - Betweenness Centralit
dibujar_influencer(G, pos, top_eigenvector, "Top Influencer - Eigenvector Centralit
```



Top Influencer - Betweenness Centrality





(e) Identificacion de comunidades

In [10]:
```
# Detectar comunidades usando particion codiciosa
communities = list(greedy_modularity_communities(G))

print("Numero de comunidades detectadas:", len(communities))
for i, comunidad in enumerate(communities, start=1):
    print(f"Comunidad {i} (tamaño {len(comunidad)}): {sorted(comunidad)}")
```

```
Numero de comunidades detectadas: 3
Comunidad 1 (tamaño 17): [8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31,
32, 33]
Comunidad 2 (tamaño 9): [1, 2, 3, 7, 9, 12, 13, 17, 21]
Comunidad 3 (tamaño 8): [0, 4, 5, 6, 10, 11, 16, 19]
```

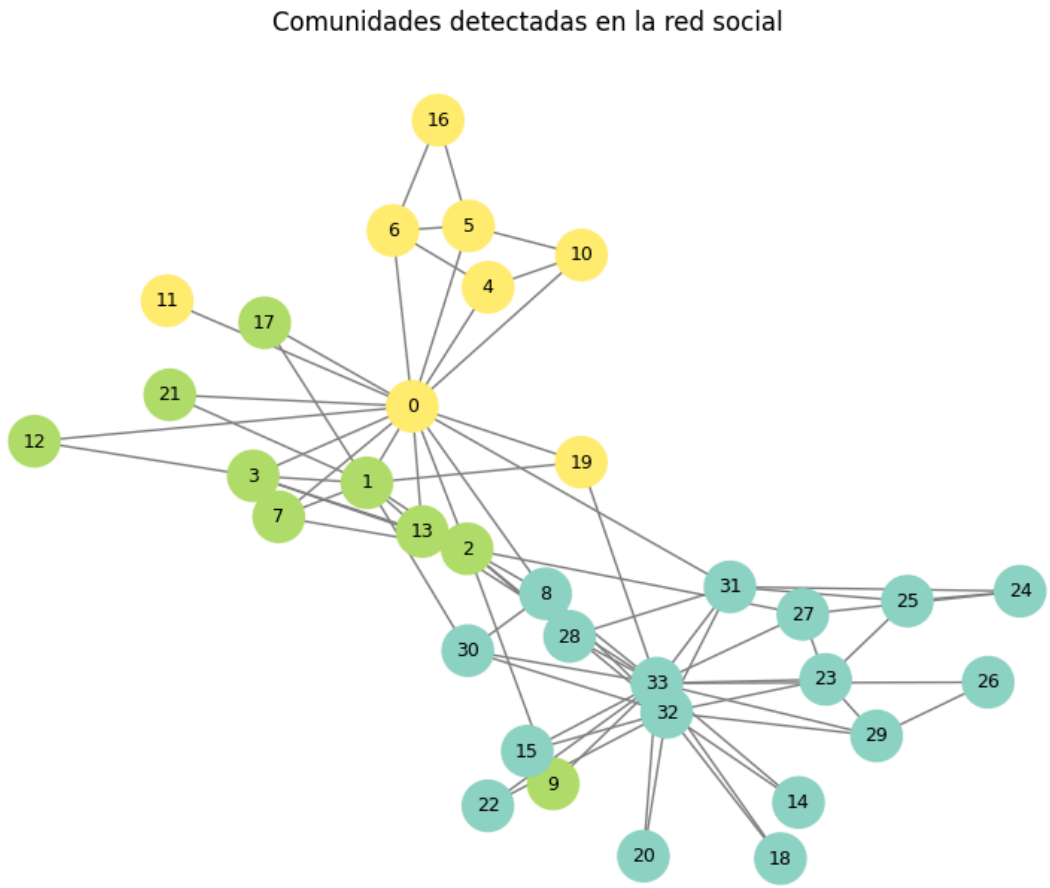
In [11]:
```
# Mostrar solo comunidades de tamano 3, 4 y 5 o mas
print("Clasificacion por tamaño:\n")

for i, comunidad in enumerate(communities, start=1):
    tam = len(comunidad)
    if tam == 3:
        print(f"Comunidad {i}: tamaño 3 -> {sorted(comunidad)}")
    elif tam == 4:
        print(f"Comunidad {i}: tamaño 4 -> {sorted(comunidad)}")
```

```
    elif tam >= 5:
        print(f"Comunidad {i}: tamaño 5 o mas -> {sorted(comunidad)}")
```

```
Clasificacion por tamaño:

Comunidad 1: tamaño 5 o mas -> [8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 3
0, 31, 32, 33]
Comunidad 2: tamaño 5 o mas -> [1, 2, 3, 7, 9, 12, 13, 17, 21]
Comunidad 3: tamaño 5 o mas -> [0, 4, 5, 6, 10, 11, 16, 19]
```

In [12]:

```
# Colorear nodos segun la comunidad a la que pertenecen
mapa_colores = {}
for i, comunidad in enumerate(communities):
    for nodo in comunidad:
        mapa_colores[nodo] = i

node_colors = [mapa_colores[n] for n in G.nodes()]

plt.figure()
nx.draw(
    G,
    pos,
    with_labels=True,
    node_color=node_colors,
    cmap=plt.cm.Set3,
    edge_color="gray",
    node_size=600,
    font_size=9
)
plt.title("Comunidades detectadas en la red social")
plt.show()
```



Cada color es una comunidad distinta. Hay entonces 3 comunidades.

In [13]:
```
for c in communities:
    print(len(c), list(c))
```

```
17 [8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33]
9 [1, 2, 3, 7, 9, 12, 13, 17, 21]
8 [0, 16, 19, 4, 5, 6, 10, 11]
```

Aplicando el algoritmo de particion codiciosa, se obtuvieron las siguientes comunidades:

Tamaño 17: {8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33}

Tamaño 9: {1, 2, 3, 7, 9, 12, 13, 17, 21}

Tamaño 8: {0, 16, 19, 4, 5, 6, 10, 11}

No se encontraron comunidades de tamaño 3 ni 4. Todas las comunidades detectadas tienen 5 o más nodos, indicando grupos de alta conexion dentro de la red.